Supplementary Model and Solver Details

This supplement accompanies *Coordinated Scheduling and Process-Mode Selection for Dual-Source Rotating-Chamber Cluster Tools*. It expands notation, physical constraints, initial-schedule construction and the learned cut selector. It contains no additional experimental outcomes.

I. Problem description and two-stage scheduling model

A. Equipment configuration and scheduling task

Given the product lot, process modes, wafer-pair layout, and initial equipment state, scheduling determines the start and finish times of loading, alignment, handling, rotation, processing, cleaning, and return actions, together with their order on shared resources. Products depart from LPs and return after processing. Dummies leave vacuum storage, perform filling or cleaning tasks, and return. Robot service, chamber occupancy, and dummy inventory couple the routes.

Inputs comprise action durations, resource capacities, residency limits, cleaning intervals, and dummy counts. Stage one minimizes the final product return time. Stage two improves waiting and cadence under the resulting makespan bound. Each feasible schedule is represented by an action sequence specifying wafers, resources, and start and completion times for performance evaluation and checks of precedence and resource conflicts.

Action durations are deterministic within a solve. No buffers are assumed beyond LPs, aligners, load locks, rotating chambers, and vacuum storage, so each wait corresponds to a material location and resource-occupancy interval.

B. PW-pair modeling for the two process modes

To represent wafer-level residency and paired handling together, shared transfer and chamber occupancy are modeled at the pair level, while alignment and route timing retain wafer-level events.

A wafer pair (PW pair) is a physical two-wafer transport and chamber-occupancy unit. Its members may be product+product (product PW pair), dummy+dummy (dummy PW pair), or product+dummy (mixed PW pair). Product wafers are preferentially paired by mode and index; an odd tail product may be paired with one dummy, while full-occupancy filling and cleaning may use dummy PW pairs. The sets $\mathcal{P}^F$ and $\mathcal{P}^L$ index product-containing PW pairs, including product PW pairs and any required mixed PW pairs. Alignment and residency remain wafer-level operations; the two members share a time window only on a selected two-hand transfer or the same chamber side. The prescribed layout specifies product-containing PW pairing and compatible chamber positions. The MILP optimizes the remaining activation and assignment decisions, physical dummy identities, paired transfers, resource order, and event times within this layout. Legal cleaning segments are generated from the cleaning interval and complete recipe boundaries, after which the MILP activates the cleaning tasks and schedules their times.

A product follows LP–ATR–AL–ATR–upper load lock–VTR–rotary chamber–VTR–lower load lock–ATR–LP: the ATR first places the wafer on the aligner, then retrieves it for transfer into the vacuum domain. A dummy travels from vacuum storage through the VTR to the chamber and returns through the VTR after use. A regular all-product 4×1 batch contains two product PW pairs, hence four product wafers, and processes all four positions synchronously once before last-in–first-out unloading; only the last incomplete batch may use a dummy or mixed PW pair to fill all positions. In 2×2, every product-containing PW pair occupies two adjacent process cycles so that each of its product members is processed twice. For three product PW pairs, a dummy PW pair D is loaded first, the chamber rotates, and P1 is loaded before D+P1 undergoes chamber processing. Chamber processing then alternates with "chamber rotation + chain slide", changing the occupancy successively to P1+P2, P2+P3, and P3+D; P1, P2, and P3 are each processed twice before all wafers are unloaded. Figures 1 and 2 show the event sequences.

Tables I and II list the sets, parameters, decision variables, fixed indicators, and derived quantities. The quantities x^F, x^L, and μ^L lie in $[0, 1]$ and are fixed to zero or one by equality constraints defining the prescribed layout.

C. Source interfaces, operation activation, and time relations

The fixed incidence parameter satisfies

$$\sum_{s \in \mathcal{S}} \chi_{is} = 1, \qquad i \in \mathcal{I}. \tag{1}$$

Equation (1) assigns each material unit to exactly one source. Setting $|\mathcal{S}| = 1$ gives the single-source tool; using two interfaces and assigning products and dummies to different interfaces gives the dual-source tool.

If $\mathcal{A}^{\text{int}}$ is the internal network and $\mathcal{A}_s$ contains release and return arcs for source s, then

$$\mathcal{A} = \mathcal{A}^{\text{int}} \cup \left(\bigcup_{s \in \mathcal{S}} \mathcal{A}_s \right). \tag{2}$$

Equation (2) means that one- and two-source tools differ only at route endpoints; processing, cleaning, shared resources, and timing use the same internal model, so a dual-source tool is not the sum of two independent single-source schedules.

Canonical pair positions satisfy

$$\begin{aligned} x^F_{pcbl} &= \bar{x}^F_{pcbl}, \quad \sum_{c,b,l} x^F_{pcbl} = 1 \quad (p \in \mathcal{P}^F), \\ x^L_{pck} &= \bar{x}^L_{pck}, \quad \sum_{c,k} x^L_{pck} = 1 \quad (p \in \mathcal{P}^L). \end{aligned} \tag{3}$$

Equation (3) assigns each wafer pair to one compatible position and fixes equivalent locations to canonical indices. Event

TABLE I
CORE NOTATION I: SETS, INDICES, AND INPUT PARAMETERS

Symbol	Domain and role	Meaning
$\mathcal{S}, \mathcal{I}$	sets/indices	Material-source interfaces and material units; $i \in \mathcal{I}$ may be a product or dummy demand.
$\mathcal{W}$	set/index	Product wafers, indexed by w.
$\mathcal{P}^F, \mathcal{P}^L$	sets/indices	Product-containing PW pairs assigned to 4×1 and 2×2; they include P+P product pairs and any required P+D mixed pairs, indexed by p.
$\mathcal{C}$	set/index	Rotary process chambers, indexed by c.
$\mathcal{B}_c, \mathcal{K}_c$	ordered sets/indices	Candidate 4×1 batches b and 2×2 product-chain positions k in chamber c.
$\mathcal{L} = \{1,2\}$	set/index	Front and back sides of a 4×1 batch.
$\mathcal{Q}_c, \mathcal{E}_c$	ordered sets/indices	2×2 process cycles q and cleaning segments e in chamber c.
$\mathcal{V}_c$	set/index	Union of all 4×1 batches and 2×2 cycles in chamber c.
$\mathcal{O}, \mathcal{A}$	sets/indices	Candidate physical operations and required process-precedence arcs.
$\mathcal{R}, \Lambda_r$	sets/indices	Shared resources and physical slots of resource r.
$\mathcal{J}_c^D, \mathcal{T}_c$	sets/indices	Dummy-wafer demands and reusable physical dummies of chamber c.
χ_{is}	known binary constant	Whether material unit i belongs to source interface s.
$\bar{x}^F_{pcbl}, \bar{x}^L_{pck}$	known binary constants	Canonical pair-to-4×1-side or pair-to-2×2-position assignment.
$q_c^-(k), q_c^+(k)$	known nonnegative integer maps	First and second process cycles of 2×2 chain position k, with $q_c^+(k) = q_c^-(k)+1$.
$\bar{z}_{ce}, \bar{a}^F_{cbe}, \bar{a}^L_{cqe}$	known binary constants	Active cleaning segments and pre-generated membership of batches or cycles.
$H, \bar{Q}_c$	nonnegative integer constants	Maximum cycles between cleanings and generated 2×2 cycles in chamber c.
$d_o, \delta_{oo'}$	nonnegative constants	Duration of operation o and minimum lag on arc (o,o').
d^F_v, d^L, r_c	nonnegative constants	4×1 stage duration, 2×2 process time, and chamber half-rotation time.
$\rho^r_{oo'}$	nonnegative constant	Empty travel or reset time from operation o to o' on resource r.
$M, \bar{R}_{wr}$	positive constants	Valid horizon-based big-M and maximum residency of wafer w in restricted region r.
κ_o, Γ_o	integer/set constants	Slots required by operation o and the slots it may use.
$\lambda_1, \ldots, \lambda_4$	nonnegative constants	Stage-two weights for total wait, maximum wait, cadence deviation, and soft-limit excess.
$\bar{\omega}_\alpha, \varepsilon_C$	nonnegative constants	Soft limit for wait α and the stage-two makespan tolerance.
$\mathcal{F}, \mathcal{D}_{\text{fix}}$	sets	Full MILP feasible region and discrete indices fixed in stage two.
$Q^w, \mathcal{I}_h$	sets/indices	Wait quantities and process starts admitting cadence second differences.

times and resource order remain optimization decisions within the prescribed layout.

A position containing a pair must be active, and candidate positions are used consecutively:

$$\begin{aligned} x^F_{pcbl} &\le u^F_{cb}, \quad u^F_{c,b+1} \le u^F_{cb}, \\ x^L_{pck} &\le u^L_{ck}, \quad u^L_{c,k+1} \le u^L_{ck}. \end{aligned} \tag{4}$$

Equation (4) activates any batch or chain position containing a product and excludes a nonphysical inactive position followed by an active one.

For each candidate operation o, activation and duration satisfy

$$0 \le s_o \le M b_o, \qquad 0 \le e_o \le M b_o, \qquad e_o = s_o + d_o b_o. \tag{5}$$

Equation (5) sets inactive operation times to zero and gives an active operation duration d_o. It applies to robot moves, alignment, pump/vent, rotation, product processing, dummy transport, and cleaning.

For every required process arc $(o,o') \in \mathcal{A}$,

$$b_{o'} \le b_o, \qquad s_{o'} \ge e_o + \delta_{oo'} - M(2 - b_o - b_{o'}). \tag{6}$$

Equation (6) activates a successor only with its predecessor and, when both exist, starts it after the predecessor and minimum lag. These arcs connect product routes, dummy routes, and load–process–unload chains.

D. Occupancy and event order in the two processing modes

For 4×1, let $A^F_{cb} = \sum_{p,l} x^F_{pcbl}$ be the number of product pairs assigned to batch (c,b). Full occupancy and dummy filling are

$$\begin{aligned} A^F_{cb} &= \sum_{p\in\mathcal{P}^F} \sum_{l\in\mathcal{L}} x^F_{pcbl}, \\ \sum_{p\in\mathcal{P}^F} x^F_{pcbl} + f^F_{cbl} &= u^F_{cb} \qquad (l \in \mathcal{L}), \\ \sum_{l\in\mathcal{L}} f^F_{cbl} &= 2u^F_{cb} - A^F_{cb}. \end{aligned} \tag{7}$$

Equation (7) gives each active 4×1 batch two sides, each occupied by one product pair or one dummy pair; all four chamber positions are therefore filled at process start.

Active batches are consecutive and dummy filling is restricted to the last active batch:

$$\begin{aligned} A^F_{cb} &\ge u^F_{cb}, & u^F_{c,b+1} &\le u^F_{cb}, \\ A^F_{cb} &\ge 2u^F_{c,b+1}, & t^F_{cb} &\le u^F_{cb}, \\ t^F_{cb} &\le 1 - u^F_{c,b+1}, & t^F_{cb} &\ge u^F_{cb} - u^F_{c,b+1}. \end{aligned} \tag{8}$$

Equation (8) requires at least one product pair in an active batch and two product pairs if a later batch is active. The last three inequalities set $t^F_{cb} = 1$ exactly at the active tail; the final candidate uses $t^F_{cB} = u^F_{cB}$.

TABLE II
CORE NOTATION II: DECISION VARIABLES, FIXED INDICATORS, AND DERIVED QUANTITIES

Symbol	Domain and role	Meaning
x^F_{pcbl}, x^L_{pck}	fixed $[0,1]$ indicators	Product-containing PW-pair placement on a 4×1 batch side or at a 2×2 chain position.
μ^L_{pcq}	fixed continuous $[0,1]$ alias	Whether 2×2 product-containing PW pair p is processed in cycle q of chamber c.
$u^F_{cb}, f^F_{cbl}, t^F_{cb}$	binary variables	4×1 batch activation, dummy filling on a side, and final active batch.
$u^L_{ck}, a^L_c, v^L_{cq}, h^L_{ck}$	binary variables	2×2 position, chain, process-cycle, and chain-tail activation.
$z_{ce}, a^F_{cbe}, a^L_{cqe}, q^{\mathrm{cl}}_{ce}$	fixed indicators/variables	Cleaning segments, task membership, and cleaning-task activation.
τ_{ce}	fixed binary parameter	Whether segment e is the final production segment and exactly exhausts H cycles, thereby triggering terminal cleaning.
$q_j, g_{j\tau}$	binary variables	Dummy demand and assignment to physical dummy τ.
$b_o, y^r_{oo'}$	binary variables	Operation activation and whether o precedes o' on resource r.
$h_{o\ell}$	binary slot variable	Whether operation o occupies physical slot ℓ.
γ_{pa}	binary variable	Whether pair p uses one paired transfer on robot route a.
s_o, e_o	nonnegative variables	Start and end times of operation o.
$s^{F,\nu}, e^{F,\nu}, s^L, e^L$	continuous time quantities	Stage-specific notation for s_o, e_o in the two modes.
A^F_{cb}, N_{ce}	derived continuous/integer quantities	Product-pair count in a 4×1 batch and active cycles in a cleaning segment.
a_{vce}	fixed binary alias	Whether task $v\in\mathcal{V}_c$ belongs to cleaning segment e.
$C_w, C_{\max}$	nonnegative variables	Product completion time and lot makespan.
$\omega_\alpha, L_{\mathrm{sum}}, L_{\max}$	nonnegative variables/derived quantities	Stage-two wait or idle quantities, their sum, and their maximum.
$\Delta_{ceq}, v_{ceq}, \eta_\alpha$	derived/nonnegative variables	Cadence second difference, its absolute-value bound, and soft wait excess.
h_{ceq}, Φ	continuous alias/derived objective	Start of the qth process in a segment and stage-two schedule-quality objective.
$\xi, \widehat{\xi}, \widehat{C}$	vector/stage constants	Current variables, stage-one incumbent, and incumbent makespan.

Let FL, BL, P, BU, FU denote front load, back load, processing, back unload, and front unload. Then

$$\begin{aligned} e^{F,\nu}_{cb} &= s^{F,\nu}_{cb} + d^F_\nu u^F_{cb} \\ &\quad \nu \in \{FL, BL, P, BU, FU\}, \\ s^{F,BL}_{cb} &\ge e^{F,FL}_{cb} + r_c - M(1-u^F_{cb}), \\ s^{F,P}_{cb} &= e^{F,BL}_{cb}, \\ s^{F,BU}_{cb} &\ge e^{F,P}_{cb} - M(1-u^F_{cb}), \\ s^{F,FU}_{cb} &\ge e^{F,BU}_{cb} + r_c - M(1-u^F_{cb}). \end{aligned} \tag{9}$$

Equation (9) loads the front side, rotates to load the back, starts synchronous processing after the back load, and then unloads the back side before rotating and unloading the front. Consecutive batches are linked by $s^{F,FL}_{c,b+1} \ge e^{F,FU}_{cb} - M(1-u^F_{c,b+1})$.

For 2×2, each product pair occupies one ordered chain position:

$$u^L_{ck} = \sum_{p\in\mathcal{P}^L} x^L_{pck}, \qquad u^L_{c,k+1} \le u^L_{ck}, \qquad a^L_c = u^L_{c1}. \tag{10}$$

Equation (10) creates a one-to-one relation between product pairs and consecutive chain positions and uses a^L_c to indicate whether chamber c contains a 2×2 chain.

Let $q^-_c(k)$ and $q^+_c(k)$ denote the first and second process cycles of chain position k. The pair–cycle incidence is

$$\begin{aligned} q^+_c(k) &= q^-_c(k) + 1, \\ \mu^L_{pcq} &= \sum_{\substack{k\in\mathcal{K}_c: \\ q\in\{q^-_c(k), q^+_c(k)\}}} x^L_{pck}, \\ \sum_{q\in\mathcal{Q}_c} \mu^L_{pcq} &= 2\sum_{k\in\mathcal{K}_c} x^L_{pck}. \end{aligned} \tag{11}$$

Equation (11) maps every assigned 2×2 product-containing PW pair to exactly two adjacent process cycles. Every product member inherits the same μ^L_{pcq} incidence, so a product is not omitted even when an odd tail uses a P+D mixed PW pair. The equation also distinguishes the modes explicitly: a 2×2 product-containing pair spans two cycles, whereas two 4×1 side pairs are processed once in one full batch.

The generated layout supplies the number $\bar{Q}_c$ of process cycles, and h^L_{ck} identifies the chain tail:

$$\begin{aligned} v^L_{cq} &= \begin{cases} a^L_c, & q \le \bar{Q}_c, \\ 0, & q > \bar{Q}_c, \end{cases} \\ \sum_k h^L_{ck} &= a^L_c, \qquad h^L_{ck} \le u^L_{ck}, \qquad h^L_{ck} + u^L_{c,k+1} \le 1, \\ h^L_{ck} &\ge u^L_{ck} - u^L_{c,k+1}. \end{aligned} \tag{12}$$

Equation (12) gives $n+1$ process cycles for a segment with n product-containing PW pairs: the head is dummy PW pair+P1, internal cycles are Pi+P$i+1$, and the tail is Pn+dummy PW pair. The final relations make h^L_{ck} one only at the last active product position.

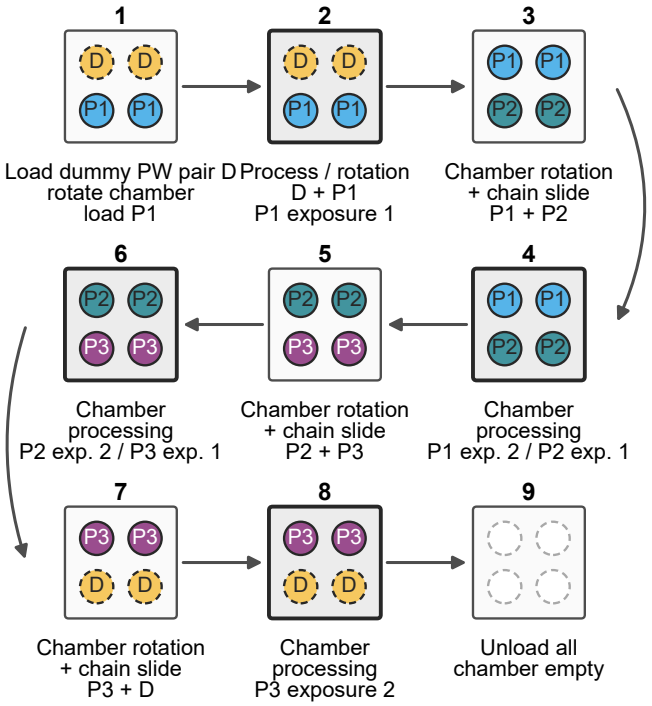


Fig. 1. 2×2 chained processing. A PW pair may be P+P, D+D, or P+D; here each Pi is a P+P product PW pair and D is a D+D dummy PW pair. Step 1 loads D, rotates the chamber, and loads P1; Step 2 performs chamber processing on D+P1; Step 3 applies chamber rotation + chain slide to obtain P1+P2. Repeating "chamber processing–chamber rotation + chain slide" makes P1, P2, and P3 participate in two adjacent process cycles, so every product wafer is processed twice before all wafers are unloaded.

For a legal chain segment $(q_0, \ldots, q_T)$, let H_0, B_q, and T_0 denote head loading, internal exchange, and tail unloading. Then

$$\begin{aligned} e^L_{cq} - s^L_{cq} &= d^L v^L_{cq}, \\ s^L_{cq_0} &= e^{H_0}_c, \\ s^B_{cq} &\geq e^L_{c,q-1} - M(1 - a^L_c), \\ s^L_{cq} = e^B_{cq} \quad (q_0 < q \leq q_T), & \\ s^{T_0}_c &\geq e^L_{cq_T} - M(1 - a^L_c). \end{aligned} \tag{13}$$

Equation (13) begins the first process immediately after head loading, performs an internal exchange only after the previous cycle, and unloads the tail only after the last process. It directly realizes dummy+P1, P1+P2, …, Pn+dummy.

E. Cleaning and reusable dummy wafers

Cleaning segments are generated from H and complete process boundaries and enter the MILP as fixed binary indicators:

$$z_{ce} = \bar{z}_{ce}, \qquad a^F_{cbe} = \bar{a}^F_{cbe} u^F_{cb}, \qquad a^L_{cqe} = \bar{a}^L_{cqe} v^L_{cq}. \tag{14}$$

Equation (14) prevents arbitrary cleaning locations: 4×1 may split only at batch boundaries and 2×2 only after a complete chain segment, while the MILP still schedules process and cleaning times.

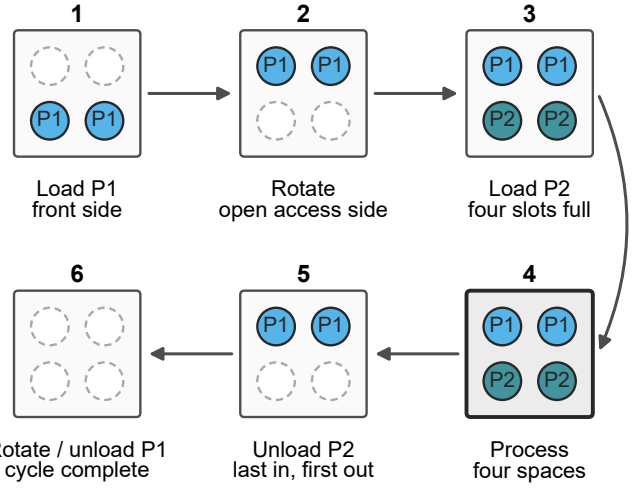


Fig. 2. 4×1 parallel processing. A regular all-product batch uses two product PW pairs, comprising four product wafers, and receives one synchronous full-occupancy process before last-in–first-out unloading; a dummy or mixed PW pair can complete the final short batch.

Let N_{ce} be the number of active process cycles in segment e:

$$\begin{aligned} N_{ce} &= \sum_b a^F_{cbe} + \sum_q a^L_{cqe}, \qquad z_{ce} \leq N_{ce} \leq H z_{ce}, \\ q^{\mathrm{cl}}_{ce} &= z_{c,e+1} + \tau_{ce}, \qquad z_{c,E_c+1} = 0. \end{aligned} \tag{15}$$

Equation (15) places at least one and at most H process cycles in an active segment. It activates an internal cleaning before every subsequent production segment and, when the final segment exactly exhausts H cycles, sets $\tau_{ce} = 1$ to activate terminal cleaning. Both are complete equipment actions; only the terminal action has no subsequent product processing.

Let $\mathcal{V}_c$ be the union of 4×1 batches and 2×2 cycles and let a_{vce} take the corresponding a^F or a^L. Cleaning order and the physical cleaning chain satisfy

$$\begin{aligned} s_v &\geq e^{\mathrm{cl}}_{c,e-1} - M(1 - a_{vce}) && (e > 1), \\ e_v &\leq s^{\mathrm{cl}}_{ce} + M(1 - a_{vce}) + M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,BL}}_{ce} &\geq e^{\mathrm{cl,FL}}_{ce} + r_c - M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,P}}_{ce} &\geq e^{\mathrm{cl,BL}}_{ce} - M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,BU}}_{ce} &\geq e^{\mathrm{cl,P}}_{ce} - M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,FU}}_{ce} &\geq e^{\mathrm{cl,BU}}_{ce} + r_c - M(1 - q^{\mathrm{cl}}_{ce}). \end{aligned} \tag{16}$$

Equation (16) starts a segment after the preceding cleaning and finishes it before the next cleaning. A cleaning task loads both chamber sides with dummies, rotates, processes, and unloads in reverse, thereby occupying the VTR, chamber, and four physical dummies rather than adding virtual downtime.

Dummy demand j is activated by a physical event:

$$q_j = \begin{cases} f^F_{cbl}, & j \text{ is a final-}4 \times 1 \text{ filling demand}, \\ a^L_c, & j \text{ is a chain endpoint demand}, \\ q^{\mathrm{cl}}_{ch}, & j \text{ is a cleaning demand}. \end{cases} \tag{17}$$

Equation (17) prevents dummies from becoming an unlimited abstract filler: a physical demand exists only for a final batch, chain head/tail, or active cleaning task.

Each active demand selects one physical dummy from inventory:

$$\sum_{\tau\in\mathcal{T}_c} g_{j\tau} = q_j, \qquad j \in \mathcal{J}_c^D. \tag{18}$$

Equation (18) assigns every active demand exactly once. Mutual exclusion subsequently orders repeated uses of the same physical dummy, so it can be reused only after unloading and return to storage.

F. Robots, load locks, chambers, and time resources

ATR, AL, VTR, and each rotary chamber are unary resources; the upper and lower load locks each have two physical slots; each dummy in inventory is a reusable slot. An operation o requiring κ_o slots satisfies

$$\sum_{\ell\in\Gamma_o} h_{o\ell} = \kappa_o b_o. \tag{19}$$

Equation (19) assigns an active operation the required number of legal physical slots and assigns none to an inactive operation. AL, ATR, VTR, and a chamber have one slot, while load locks and dummy inventory retain physical identities.

If operations o and o' may compete for slot ℓ of resource r, then

$$\begin{aligned} s_{o'} &\ge e_o + \rho^r_{oo'} - M(1-y^r_{oo'}) - M(1-h_{o\ell}) - M(1-h_{o'\ell}),\\ s_o &\ge e_{o'} + \rho^r_{o'o} - My^r_{oo'} - M(1-h_{o\ell}) - M(1-h_{o'\ell}). \end{aligned} \tag{20}$$

Equation (20) makes $y^r_{oo'}$ choose one order whenever two actions use the same slot and includes empty robot travel or equipment reset through $\rho^r_{oo'}$. It applies to ATR, AL, both load locks, VTR, rotary chambers, and individual dummies, preventing concurrent robot actions and reuse before release.

An individual product or dummy stage must coincide with its selected batch, chain position, or cleaning window. For selection x and reference stage $[s^{\mathrm{ref}}, e^{\mathrm{ref}}]$,

$$\begin{aligned} -M(1-x) \le s_o - s^{\mathrm{ref}} \le M(1-x),\\ -M(1-x) \le e_o - e^{\mathrm{ref}} \le M(1-x). \end{aligned} \tag{21}$$

Equation (21) synchronizes the individual action with its physical stage when $x = 1$ and releases the candidate relation when $x = 0$, connecting pair positions, product routes, dummy demands, and chamber windows.

If pair p uses one paired transfer on robot route a, the two wafer stages are synchronized:

$$\begin{aligned} -M(1-\gamma_{pa}) \le s_{w_1a} - s_{w_2a} \le M(1-\gamma_{pa}),\\ -M(1-\gamma_{pa}) \le e_{w_1a} - e_{w_2a} \le M(1-\gamma_{pa}). \end{aligned} \tag{22}$$

Equation (22) means that $\gamma_{pa} = 1$ moves a complete pair in one non-preemptive robot action. If $\gamma_{pa} = 0$, the wafers still occupy the same robot separately and are ordered by Eq. (20).

For the entry and exit events of wafer w in restricted equipment region r,

$$0 \le s_{\mathrm{out}(w,r)} - e_{\mathrm{in}(w,r)} \le \overline{R}_{wr}. \tag{23}$$

Equation (23) limits the physical stay in an upper load lock, chamber, lower load lock, or restricted transport region. Load-lock occupation also spans placement, pump/vent, opposite-side pickup, and pressure reset, so placement by a robot is not mistaken for slot release.

G. Makespan-first two-stage optimization

Let C_w be the time at which product wafer w completes the lower-load-lock return and reaches its load port. Stage one solves

$$C_w = e_{w,\mathrm{return}}, \qquad C_{\max} \ge C_w \ \ (w \in \mathcal{W}), \qquad \min_{\xi\in\mathcal{F}} C_{\max}. \tag{24}$$

Equation (24) measures makespan at the last product wafer's return to LP after its complete end-to-end route. Internal cleaning affects $C_{\max}$ through resource occupancy and precedence that delay subsequent product returns. Terminal cleaning remains in the action sequence, but its completion time does not directly lower-bound $C_{\max}$. It can still affect other chambers' product returns through competition for the shared VTR; this indirect effect is determined by the complete schedule. WPH uses product-return makespan as its denominator. The feasible region $\mathcal{F}$ is defined by Eqs. (1)–(23) expanded over all applicable indices.

Given stage-one incumbent $\widehat{\xi}$ and makespan $\widehat{C}$, stage two fixes the discrete structure indexed by $\mathcal{D}_{\mathrm{fix}}$ and minimizes linear schedule quality within the makespan tolerance:

$$\begin{aligned} \xi_d = \widehat{\xi}_d \quad (d \in \mathcal{D}_{\mathrm{fix}}), \qquad C_{\max} \le \widehat{C} + \varepsilon_C,\\ L_{\mathrm{sum}} = \sum_{\alpha\in\mathcal{Q}^w} \omega_\alpha, \qquad L_{\max} \ge \omega_\alpha,\\ \Delta_{ceq} = (h_{ce,q+1} - h_{ceq}) - (h_{ceq} - h_{ce,q-1}),\\ v_{ceq} \ge \Delta_{ceq}, \qquad v_{ceq} \ge -\Delta_{ceq},\\ \eta_\alpha \ge \omega_\alpha - \bar{\omega}_\alpha, \qquad \eta_\alpha \ge 0,\\ \min \Phi = \lambda_1 L_{\mathrm{sum}} + \lambda_2 L_{\max} + \lambda_3 \sum_{(c,e,q)\in\mathcal{I}_h} v_{ceq} + \lambda_4 \sum_{\alpha\in\mathcal{Q}^w} \eta_\alpha. \end{aligned} \tag{25}$$

Equation (25) preserves the stage-one makespan bound and fixes the principal discrete routes and resource orders. Retained paired-transfer choices and continuous times can change to reduce total wait, maximum wait, and variation between consecutive processing intervals within each segment. Optional soft-limit excess can also be penalized. The experiments use $(\lambda_1, \lambda_2, \lambda_3) = (1, 10, 25)$ and $\varepsilon_C = 10^{-6}$. With the soft waiting limit disabled, no excess variables are generated. For a time-limited first stage, the best incumbent value $\widehat{C}$ supplies the second-stage bound; variables in $\mathcal{D}_{\mathrm{fix}}$ are fixed while designated paired-transfer variables remain adjustable.

II. Structure-aware MILP solution framework

The scheduling model couples discrete resource assignments with continuous event times. Primal feasibility requires coordination of full occupancy, dual-source logistics, and shared-resource order, whereas dual-bound improvement depends on valid inequalities acting on discrete choices, continuous times, and their coupling. We exploit these structures through a structure-preserving binary skeleton (SPBS) for initial feasible scheduling and RDMCT-HBS for cut selection. SPBS supplies a feasible starting point, and RDMCT-HBS determines cut counts and insertion order. SCIP generates valid cuts, checks feasibility, maintains bounds, and certifies optimality [1], [2].

The solution process comprises initial-schedule construction and validation, candidate-cut generation, count prediction,

ordered-subset search, and role-submodular completion, followed by continued branch-and-bound search. All selected cuts are valid inequalities for the original model and retain their coefficients. Cut selection therefore controls the strengthening of the linear relaxation while preserving the original integer feasible set.

A. SPBS initial solution

SPBS first constructs a discrete skeleton from the recipe and equipment layout, then solves for event times in a restricted MILP. Product wafers are paired in input order. The 4×1 pairs are mapped sequentially to chambers, batches, and front/back positions, whereas the 2×2 pairs are assigned round-robin to chamber-chain positions. This determines batch-use, tail-batch, used-position, and last-position indicators. Cleaning counts partition each chamber sequence into process segments: 4×1 batches are assigned to their cleaning epochs, while a 2×2 chain ends its current segment with process-enabling carriers (PECs) when the cleaning threshold is reached and resumes after cleaning with a new PEC-bounded head. The skeleton therefore determines what is processed, in which chamber, and at which legal cleaning boundary; detailed PEC occupancy and all continuous event times remain governed by the MILP.

At the resource level, SPBS derives a conservative order from the same physical route. ATR first schedules 4×1 inputs and returns across chambers and stages the first 2×2 pair of a chamber immediately after that chamber's final full batch. It then advances chain inputs and returns segment by segment. Deterministic odd/even load-lock slots break slot symmetry, and the VTR order is aligned with ATR batch progression and cleaning actions. Let $\mathcal{F}_q$ be the binary variables fixed in relaxation level q, with skeleton value $\widehat{x}_j$. The restricted feasibility subproblem imposes

$$\ell_j^{(q)} = u_j^{(q)} = \widehat{x}_j, \qquad j \in \mathcal{F}_q \tag{26}$$

Equation (26) fixes only the skeleton choices at the current level. Start, completion, and waiting times, together with resource orders not fixed at that level, remain free.

SPBS progressively relaxes fixed-variable conditions through five levels, with nested fixed sets $\mathcal{F}_1 \supseteq \mathcal{F}_2 \supseteq \cdots \supseteq \mathcal{F}_5$. For unchanged skeleton values, releasing these conditions expands the restricted feasible set. If a level yields no feasible solution within its allocated budget, the search proceeds to the next level. The first level fixes product and batch assignments, activation states, cleaning segments, load-lock slots, and ATR, AL, LL, and VTR orders. The second releases VTR order; the third also releases load-lock order; the fourth releases all remaining resource orders while retaining structural assignments and slots; and the fifth retains only assignments, activation states, and cleaning segments. For instances with at least 32 wafers, the search starts at the fourth or fifth level. Under a tight waiting-time limit, the original feasible set without additional skeleton restrictions is also considered.

Each level solves a restricted feasibility subproblem with a zero objective and terminates at the first feasible solution. These subproblems use feasibility emphasis, aggressive heuristics, and fast presolve, without cut separation. The resulting principal discrete decisions and continuous event times are then retained in the full model while auxiliary variables associated with schedule stability are completed. A schedule satisfying all original-model constraints becomes the initial incumbent of the main problem. The main problem restores the original variable bounds and thus searches the full original feasible set. In the experiments, SPBS schedules are constructed offline and checked against their corresponding models; the PDI integration interval and main-solve timing both exclude offline initial-schedule construction.

B. HEM baseline and role-aware representation

Prior learning-based cut selectors address generic candidate sets with reinforcement learning, learned ranking, and look-ahead imitation [3]–[5]. HEM further identifies three coupled decisions—which cuts, how many cuts, and in what order—and handles them with a high-level count policy and a low-level sequence policy [6]. We retain HEM's hierarchical decomposition and pointer decoding and use its 13D sequence model as the direct baseline. Training instances, budgets, random seeds, candidate limits, and hidden dimensions are shared while representation, decoding, and post-selection completion are compared. HEM was subsequently extended to hierarchical sequence/set models [7]; HEM-13D here is a controlled reproduction of the 2023 sequence model.

The 13D HEM candidate vector comprises objective parallelism, efficacy, normalized support, integral-support ratio, normalized violation, the mean/maximum/minimum/standard deviation of row coefficients, and the corresponding four objective-coefficient statistics. These features describe immediate LP quality and numerical scale. They do not distinguish two similarly efficacious rows when one acts on discrete route choices and the other on continuous event times. In the present model, that distinction corresponds directly to changing a process/resource combination versus tightening the timing of an existing combination.

Starting from the 13 generic HEM features, variables are partitioned by their mathematical attributes into binary, general integer, objective-active continuous, continuous auxiliary, and other roles. The coefficient mass of role g in cut i is

$$p_{ig} = \frac{\sum_{j:x_j\in g} |a_{ij}|}{\sum_j |a_{ij}| + \varepsilon}. \tag{27}$$

Equation (27) is the fraction of absolute cut-coefficient mass carried by a mathematical variable role. Beyond the five role masses, the added features are mass on variables with two finite global bounds, positive-coefficient mass, discrete–continuous coupling $4(p_{iB} + p_{iI})(p_{iC_{\text{obj}}} + p_{iC_{\text{aux}}})$, the largest non-residual role mass, and normalized role entropy. The resulting vector has 23 dimensions. Role assignment depends only on variable types, objective coefficients, and global bounds. The representation is therefore invariant to variable renaming that preserves these attributes and the cut coefficients.

HEM uses a hierarchical "count policy + ordered-subset policy" decomposition: its high level predicts the number of cuts from a sequence representation, while its low level uses an LSTM encoder–decoder with pointer attention to

select cuts sequentially [6], [8]. Building on this hierarchy, the proposed selector introduces a role-aware grouped gated encoder before the LSTM, a role-submodular completion step after the pointer sequence, and width-three beam search during formal inference, which together form the complete RDMCT-HBS selection pipeline.

C. Role-aware encoding and hierarchical sequence decoding

Let m_t be the number of candidates retained at separation round t. For candidate cut i, write the 23D input as $x_i = [x_i^H; x_i^R]$, where $x_i^H \in \mathbb{R}^{13}$ contains the generic HEM features and $x_i^R \in \mathbb{R}^{10}$ contains the proposed role-aware features. The full vector is first layer-normalized; the two feature groups are then projected separately into the common hidden space and fused by an elementwise gate generated from all 23 features:

$$\begin{aligned} \widetilde{x}_i &= \mathrm{LN}(x_i), \\ u_i^H &= W_H \widetilde{x}_i^H + b_H, \qquad u_i^R = W_R \widetilde{x}_i^R + b_R, \\ g_i &= \sigma(W_g \widetilde{x}_i + b_g), \qquad z_i = g_i \odot u_i^H + (1 - g_i) \odot u_i^R. \end{aligned} \tag{28}$$

Here $z_i \in \mathbb{R}^{128}$ is the fused candidate embedding. Unlike a single projection of the 23D vector, the two branches preserve generic LP quality and mathematical-role structure separately, while g_i determines their candidate-specific contribution in each hidden dimension. Thus two cuts with similar efficacy but different concentration on discrete route variables or continuous timing variables can remain distinguishable. This grouped gated adapter is a structural change to HEM's input representation introduced in this work.

The fused candidates, in the deterministic order established by Step 1, are passed through a single-layer LSTM encoder:

$$(h_i, c_i) = \mathrm{LSTM}(z_i, h_{i-1}, c_{i-1}), \qquad i = 1, \ldots, m_t. \tag{29}$$

The states h_i turn local cut descriptors into context-dependent candidate representations and are used by both the high-level retained-count policy and the low-level ordered selector. A 128D hidden state is used to encode the ordered candidates into a contextual sequence representation. Fixing the input order also makes the encoded sequence reproducible for an identical candidate pool.

At low-level decoding step r, the decoder state d_r and all encoder states h_i are used for one glimpse attention followed by pointer attention. Let $M_{ri} = -\infty$ if candidate i already appears in the prefix and $M_{ri} = 0$ otherwise. Then

$$\begin{aligned} a_{ri} &= v_g^\top \tanh(W_e h_i + W_d d_r) + M_{ri}, \\ \alpha_{ri} &= \mathrm{softmax}_i(a_{ri}), \\ \bar{h}_r &= \sum_i \alpha_{ri} h_i, \\ b_{ri} &= v_p^\top \tanh(U_e h_i + U_g \bar{h}_r) + M_{ri}, \\ P(i_r = i \mid i_{1:r-1}, S_t) &= \mathrm{softmax}_i(b_{ri}). \end{aligned} \tag{30}$$

Equation (30) makes the sequential dependence explicit. The selected prefix updates the decoder state and the next conditional distribution. Masking excludes previously selected cuts, and the glimpse summarizes the candidate pool for the current prefix. The resulting sequence model combines contextual candidate encoding, prefix-aware pointer decoding, and the proposed representation and completion stages into one role-aware selector.

Training keeps stochastic sampling from the policy for exploration. During formal inference, instead of committing to one path at every step, the method maintains $B = 3$ partial sequences. For parent beam $b = (i_1, \ldots, i_{r-1})$ and an unselected extension j, the accumulated score is

$$L_r(b \oplus j) = L_{r-1}(b) + \log P(i_r = j \mid b, S_t), \tag{31}$$

All parent-beam–candidate extensions are compared and the three largest L_r values are retained. After the high-level target length k_t is reached, the best sequence is passed to the completion stage. This beam procedure preserves multiple high-probability partial sequences during decoding, reduces early commitment, and works jointly with the subsequent policy-anchored role-submodular completion to inject variable-role structure into the final selected set.

D. Stepwise selection from candidate pool to ordered cuts

At separation round t, the method executes the following sequence.

Step 1: form the candidate pool. SCIP first generates and validates candidate rows C_t with its own separators. To control the candidate-state dimension and evaluation cost, at most 128 candidates are retained in descending order of cut efficacy. Ties are broken by original index for deterministic input. Forced cuts remain under solver control and are not placed in the learned ranking.

Step 2: compute and encode the state. The 13 HEM and 10 role-aware features are extracted for every row, together with role masses, dominant role, and structural entropy. Equation (28) then performs grouped gated fusion and Eq. (29) produces contextual candidate states. The encoded candidate set is S_t.

Step 3: determine the retained count. The high-level policy outputs a tanh-bounded action in $[-1, 1]$, which is mapped to a retained fraction ρ_t. The fraction is then clipped by the global 0.2 cap, SCIP's round-specific limit, and the limit of 16 cuts, producing k_t. When enough candidates exist, at least two rows are retained to maintain a nonempty selected subset in each round.

Step 4: construct the candidate order. At low-level step r, the pointer policy conditions on S_t and the selected prefix $i_{1:r-1}$ to score all remaining candidates. Selected indices are masked with negative infinity and cannot be repeated. The joint high- and low-level policy is

$$\pi_\theta(k_t, i_{1:k_t} \mid S_t) = \pi_\theta^H(k_t \mid S_t) \prod_{r=1}^{k_t} \pi_\theta^L(i_r \mid S_t, i_{1:r-1}). \tag{32}$$

Equation (32) means that the high level determines the sequence length and the low level then chooses distinct candidates within that length. Each earlier cut changes both the remaining action set and the decoder state used by later probabilities.

Step 5: beam decoding. The low-level policy samples stochastically during training. At test time, beam search of width three is used. At step r, every partial sequence is extended by every

unselected candidate; joint log-probabilities are accumulated, and the three highest-scoring items over all parent-beam–next-candidate pairs are retained. After k_t steps, the best sequence is returned. Unlike one-step greedy decoding, the beam retains an initially lower-ranked prefix whose later joint probability can be higher. This fixed-width neural construction follows the broader use of beam decoding in neural combinatorial optimization [9].

Step 6: role-submodular completion. A sequence based only on joint probability can still repeatedly cover one variable role. The first $\lceil 0.25k_t \rceil$ pointer candidates are retained as policy anchors. The remaining policy order is merged with a heuristic tail sorted by instantaneous structural quality q_i, and at most $2k_t$ rows form expansion pool $\mathcal{R}_t$. For $S \subseteq \mathcal{R}_t$, define

$$F_t(S) = \sum_{i \in S} (0.35q_i + 0.20r_i) + 0.25 \sum_{g \in \mathcal{G}} d_g \sqrt{\sum_{i \in S} p_{ig}} + \frac{0.20}{|\mathcal{R}_t|} \sum_{u \in \mathcal{R}_t} \max_{i \in S} s_{ui}, \tag{33}$$

where r_i is the normalized pointer-rank prior, d_g is the demand share of role g in the expansion pool, and s_{ui} is clipped cosine similarity on generic-quality and role-mass descriptors. The score q_i combines efficacy, violation, integral support, recognized-role ratio, dominant-role ratio, and structural entropy. In Eq. (33), the first term retains instantaneous quality and the learned order; the square-root coverage term gives diminishing marginal gain to repeated role mass; and the final term makes the selected cuts represent the main structures in the pool. Starting from the anchors, the method recomputes the marginal gain of every unselected candidate and adds the largest until k_t cuts have been selected. With nonnegative weights, role masses, and clipped similarities, and $F_t(\varnothing) = 0$, the modular, concave-coverage, and facility-location terms form a monotone submodular objective. For fixed anchors A_t and expansion pool, define $f(U) = F_t(A_t \cup U) - F_t(A_t)$. Greedy completion under the cardinality constraint has the standard $1 - 1/e$ approximation property for this residual objective [10]. This ratio concerns coverage under the residual surrogate objective within the fixed pool; PDI and solution time are evaluated experimentally.

The k_t selected cuts are added to the current LP in anchor order followed by greedy-completion order. Candidate truncation, state encoding, count prediction, sequence decoding, and submodular completion jointly determine the ordered cut set used to strengthen the relaxation in the current round.

Algorithm 1: Cut selection by parallel hierarchical REINFORCE and beam search

Algorithm steps

Input: SCIP-validated rows C_t, high-level policy π_θ^H, low-level pointer policy π_θ^L, beam width $B = 3$, and the current SCIP cut limit.
Output: Ordered cut set $S_t^\star = (i_1, \ldots, i_{k_t})$ to be added to the current LP.
1. Train the policies. Two workers collect rollouts in parallel with terminal reward $R = -\text{PDI}$; a central learner applies hierarchical REINFORCE with separate EMA baselines and synchronous Adam updates. Fix the parameters for formal inference.
2. Form the pool. Sort by efficacy, break ties by original index, and retain at most 128 rows.
3. Encode the state. Compute 13 HEM features, ten role-aware features, and role masses; apply grouped projections, sigmoid-gated fusion, and LSTM encoding.
4. Determine the count. Predict the retained fraction with π_θ^H and obtain k_t after the 0.2 fraction cap, SCIP round limit, and 16-cut limit.
5. Initialize the beam. Set $\mathcal{B}_0$ to the empty sequence with log probability zero.
6. Expand. For $r = 1, \ldots, k_t$, obtain masked conditional probabilities from one glimpse plus pointer attention, extend every prefix in $\mathcal{B}_{r-1}$ by every unselected candidate, and accumulate log probability.
7. Prune. Retain the three highest joint-probability parent–candidate extensions as $\mathcal{B}_r$; after the loop, take the leading sequence in $\mathcal{B}_{k_t}$.
8. Fix anchors. Retain its first $\lceil 0.25k_t \rceil$ rows as policy anchors A_t.
9. Form the expansion pool. Merge the remaining pointer order with structural-quality order, remove duplicates, and keep at most $2k_t$ rows as $\mathcal{R}_t$.
10. Complete submodularly. Set $S \leftarrow A_t$; while $|S| < k_t$, add the $i \in \mathcal{R}_t \setminus S$ maximizing $F_t(S \cup \{i\}) - F_t(S)$.
11. Add cuts. Add $S_t^\star$ to the current LP in anchor order followed by completion order.

E. Policy training and exact solution

One time-limited SCIP solve forms a rollout, with its terminal PDI used as the cost. The current policy is executed in every eligible cut-selection round. Training uses the high-level fraction and low-level sampled sequence at the first valid candidate set of that rollout as its action sample, and the PDI of the complete solve is returned to both policies. Writing $R = -\text{PDI}$ as a reward, the low-level estimator is

$$\widehat{\nabla_\theta J} = \frac{1}{B} \sum_{b=1}^{B} (R_b - \bar{R}_{\text{EMA}}) \nabla_\theta \log \pi_\theta(\tau_b), \qquad \bar{R}_{\text{EMA}} \leftarrow 0.9\bar{R}_{\text{EMA}} + 0.1\bar{R}. \tag{34}$$

Equation (34) gives the batch policy-gradient update formed from the first eligible candidate-set action sample and an exponential-moving-average baseline; B is the rollout batch and τ_b is the cut sequence sampled at the first eligible candidate set in rollout b. The equivalent loss is PDI-cost advantage times log probability. Low-level log probability is averaged over sequence length so that a longer cut sequence does not produce a larger gradient merely because it contains more terms. The high-level policy uses the same PDI cost and an independent EMA baseline and is updated every two epochs; both gradient norms are clipped at 2. Parallel environments collect rollouts, and a central learner updates both policies synchronously after each batch, forming a parallel hierarchical REINFORCE procedure [11], [12].

Training uses five instances and five random seeds. Each seed has two workers, each completing four 30-s/1024-MB rollouts per epoch, for eight rollouts per epoch and 60 epochs. Initial Adam learning rates are 10^{-4} for the low-level policy and 5×10^{-4} for the high-level policy and are multiplied by 0.96 every ten epochs. Training uses stochastic decoding and formal inference uses beam width three. Every configuration–seed combination uses the fixed epoch-60 parameters without benchmark- or test-set parameter selection. HEM-13D, feature-only 23D, and the complete 23D method are trained separately

so that weights are never mixed across input semantics or completion environments.

RDMCT-HBS only orders and selects valid rows already generated by SCIP, and SPBS submits only solutions checked in the complete model. Neither changes the objective, constraints, integrality, or bound logic. The learned components therefore change the search path and finite-time bound convergence, whereas feasibility and optimality retain the same meaning as in default SCIP.

AI Assistance

OpenAI ChatGPT and Codex assisted with language revision, translation, LaTeX preparation and explanation of the model and solver code. The mathematical formulation and previously reported numerical evidence were preserved.

References

[1] T. Achterberg, "SCIP: Solving constraint integer programs," *Mathematical Programming Computation*, vol. 1, pp. 1–41, 2009.

[2] K. Bestuzheva, M. Besançon, W. K. Chen, A. Chmiela, T. Donkiewicz, J. van Doornmalen, L. Eifler, O. Gaul, G. Gamrath, A. Gleixner, L. Gottwald, C. Graczyk, K. Halbig, A. Hoen, C. Hojny, R. van der Hulst, T. Koch, M. E. Lübbecke, S. J. Maher, F. Matter, E. Mühmer, B. Müller, M. E. Pfetsch, F. Schlösser, F. Serrano, Y. Shinano, B. Sofranac, M. Turner, S. Vigerske, F. Wegscheider, P. Wellner, D. Weninger, and J. Witzig, "Enabling research through the SCIP optimization suite 8.0," *ACM Transactions on Mathematical Software*, vol. 49, pp. 22:1–22:21, 2023.

[3] Y. Tang, S. Agrawal, and Y. Faenza, "Reinforcement learning for integer programming: Learning to cut," in *Proceedings of the 37th International Conference on Machine Learning*. PMLR, 2020, pp. 9367–9376. [Online]. Available: https://proceedings.mlr.press/v119/tang20a.html

[4] Z. Huang, K. Wang, F. Liu, H. L. Zhen, W. Zhang, M. Yuan, J. Hao, Y. Yu, and J. Wang, "Learning to select cuts for efficient mixed-integer programming," *Pattern Recognition*, vol. 123, p. 108353, 2022.

[5] M. B. Paulus, G. Zarpellon, A. Krause, L. Charlin, and C. Maddison, "Learning to cut by looking ahead: Cutting plane selection via imitation learning," in *Proceedings of the 39th International Conference on Machine Learning*. PMLR, 2022, pp. 17 584–17 600. [Online]. Available: https://proceedings.mlr.press/v162/paulus22a.html

[6] Z. Wang, X. Li, J. Wang, Y. Kuang, M. Yuan, J. Zeng, Y. Zhang, and F. Wu, "Learning cut selection for mixed-integer linear programming via hierarchical sequence model," in *Proceedings of the Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=Zob4P9bRNcK

[7] J. Wang, Z. Wang, X. Li, Y. Kuang, Z. Shi, F. Zhu, M. Yuan, J. Zeng, Y. Zhang, and F. Wu, "Learning to cut via hierarchical sequence/set model for efficient mixed-integer programming," *arXiv preprint arXiv:2404.12638*, 2024. [Online]. Available: https://arxiv.org/abs/2404.12638

[8] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Advances in Neural Information Processing Systems*, 2015, pp. 2692–2700.

[9] J. Choo, Y. D. Kwon, J. Kim, J. Jae, A. Hottung, K. Tierney, and Y. Gwon, "Simulation-guided beam search for neural combinatorial optimization," in *Advances in Neural Information Processing Systems*, 2022.

[10] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions—i," *Mathematical Programming*, vol. 14, pp. 265–294, 1978.

[11] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, pp. 229–256, 1992.

[12] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proceedings of the 33rd International Conference on Machine Learning*. PMLR, 2016, pp. 1928–1937. [Online]. Available: https://proceedings.mlr.press/v48/mniha16.html